

CIS 6270

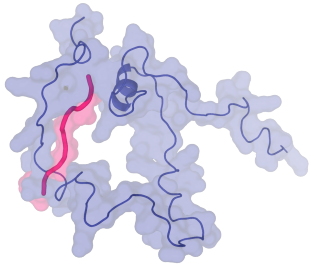
Module 5: Discrete Flow Matching

Lecture 5.4: Guidance and Multi-Objective Refinement

Thu, Nov 5, 2026

Pranam Chatterjee, PhD
Assistant Professor of Bioengineering
Assistant Professor of Computer and Information Science
University of Pennsylvania

Penn Engineering
University of Pennsylvania



Now guide samples toward the properties we need

Now guide samples toward the properties we need

When we design peptides, we need to optimize properties such as target binding, solubility, and serum half-life.

Now guide samples toward the properties we need

When we design peptides, we need to optimize properties such as target binding, solubility, and serum half-life.

Retraining on the subset that has every property is not possible, because that subset is usually empty.

Now guide samples toward the properties we need

When we design peptides, we need to optimize properties such as target binding, solubility, and serum half-life.

Retraining on the subset that has every property is not possible, because that subset is usually empty.

We can keep the trained generator and use a property model to guide sampling.

Could we change the jump rates to favor sequences with the property we want?

Discrete Guidance

UNLOCKING GUIDANCE FOR DISCRETE STATE-SPACE DIFFUSION AND FLOW MODELS

Hunter Nisonoff,* Junhao Xiong,* Stephan Allenspach,* Jennifer Listgarten
University of California, Berkeley
{hunter_nisonoff,junhao_xiong,stephall,jennl}@berkeley.edu

Nisonoff,..,Listgarten. *ICLR* (2025)

First, write down the distribution we actually want

First, write down the distribution we actually want

The target is Bayes' rule applied to the data distribution:

$$p(x_1 \mid y) = \frac{p(y \mid x_1)\, p(x_1)}{\sum_{x'} p(y \mid x')\, p(x')}$$

First, write down the distribution we actually want

The target is Bayes' rule applied to the data distribution:

$$p(x_1 \mid y) = \frac{p(y \mid x_1)\, p(x_1)}{\sum_{x'} p(y \mid x')\, p(x')}$$

The conditional distribution of the data given the property y is the likelihood of y given the sequence, times the model probability, divided by the sum of that product over all sequences.

First, write down the distribution we actually want

The target is Bayes' rule applied to the data distribution:

$$p(x_1 \mid y) = \frac{p(y \mid x_1)\, p(x_1)}{\sum_{x'} p(y \mid x')\, p(x')}$$

The conditional distribution of the data given the property y is the likelihood of y given the sequence, times the model probability, divided by the sum of that product over all sequences.

The normalizer requires a sum over every sequence. We still need a way to sample without computing that sum.

First, write down the distribution we actually want

The target is Bayes' rule applied to the data distribution:

$$p(x_1 \mid y) = \frac{p(y \mid x_1)\, p(x_1)}{\sum_{x'} p(y \mid x')\, p(x')}$$

The conditional distribution of the data given the property y is the likelihood of y given the sequence, times the model probability, divided by the sum of that product over all sequences.

The normalizer requires a sum over every sequence. We still need a way to sample without computing that sum.

With Discrete Guidance, we write the rates of the conditioned process in closed form, with no sum over sequences.

Now let c be the property we want

Now let c be the property we want

With c the desired property:

$$u_t(y \mid x,\, c) = \frac{p(c \mid y,\, t)}{p(c \mid x,\, t)}\, u_t(y, x), \qquad y \neq x$$

Now let c be the property we want

With c the desired property:

$$u_t(y \mid x,\, c) = \frac{p(c \mid y,\, t)}{p(c \mid x,\, t)}\, u_t(y, x), \qquad y \neq x$$

The guided rate from x to y equals the probability of the property c given the destination y, divided by its probability given the current state x, times the unguided rate from x to y.

Now let c be the property we want

With c the desired property:

$$u_t(y \mid x,\, c) = \frac{p(c \mid y,\, t)}{p(c \mid x,\, t)}\, u_t(y, x), \qquad y \neq x$$

The guided rate from x to y equals the probability of the property c given the destination y, divided by its probability given the current state x, times the unguided rate from x to y.

We increase rates toward states with higher property scores and decrease rates toward states with lower scores.

Now let c be the property we want

With c the desired property:

$$u_t(y \mid x,\, c) = \frac{p(c \mid y,\, t)}{p(c \mid x,\, t)}\, u_t(y, x), \qquad y \neq x$$

The guided rate from x to y equals the probability of the property c given the destination y, divided by its probability given the current state x, times the unguided rate from x to y.

We increase rates toward states with higher property scores and decrease rates toward states with lower scores.

We evaluate the property model on noisy intermediate states, so we train it on noisy states too.

Now let c be the property we want

With c the desired property:

$$u_t(y \mid x,\, c) = \frac{p(c \mid y,\, t)}{p(c \mid x,\, t)}\, u_t(y, x), \qquad y \neq x$$

The guided rate from x to y equals the probability of the property c given the destination y, divided by its probability given the current state x, times the unguided rate from x to y.

We increase rates toward states with higher property scores and decrease rates toward states with lower scores.

We evaluate the property model on noisy intermediate states, so we train it on noisy states too.

We rebuild the diagonal as minus the row sum, so the tilted rates are still a velocity.

Define the function in that ratio

Define the function in that ratio

Define the function we use as a tilt:

$$\boxed{h_t(x) := p(c \mid x_t = x) = \mathbb{E}\big[\, p(c \mid X_1) \,\big|\, X_t = x\big]}$$

Define the function in that ratio

Define the function we use as a tilt:

$$\boxed{h_t(x) := p(c \mid x_t = x) = \mathbb{E}\big[\, p(c \mid X_1) \,\big|\, X_t = x \big]}$$

The function $h_t(x)$ is the probability of the terminal property conditional on state x at time t. Equivalently, it is the expected terminal property likelihood given the current state.

Define the function in that ratio

Define the function we use as a tilt:

$$\boxed{h_t(x) := p(c \mid x_t = x) = \mathbb{E}\big[\, p(c \mid X_1) \,\big|\, X_t = x \big]}$$

The function $h_t(x)$ is the probability of the terminal property conditional on state x at time t. Equivalently, it is the expected terminal property likelihood given the current state.

It is the probability of the property at the end, starting from the current state.

Define the function in that ratio

Define the function we use as a tilt:

$$\boxed{h_t(x) := p(c \mid x_t = x) = \mathbb{E}\big[\, p(c \mid X_1) \,\big|\, X_t = x \big]}$$

The function $h_t(x)$ is the probability of the terminal property conditional on state x at time t. Equivalently, it is the expected terminal property likelihood given the current state.

It is the probability of the property at the end, starting from the current state.

That function satisfies the backward equation of the unguided process, so it is space-time harmonic.

Define the function in that ratio

Define the function we use as a tilt:

$$\boxed{h_t(x) := p(c \mid x_t = x) = \mathbb{E}\big[\, p(c \mid X_1) \,\big|\, X_t = x \big]}$$

The function $h_t(x)$ is the probability of the terminal property conditional on state x at time t. Equivalently, it is the expected terminal property likelihood given the current state.

It is the probability of the property at the end, starting from the current state.

A partly generated sequence is useful when its possible completions are likely to satisfy the goal.

Recall Doob’s h-transform for a jump process

Recall Doob's h-transform for a jump process

With Doob's h-transform of a jump process, we tilt every rate by a ratio of h:

$$\boxed{u_t^h(y, x) = \frac{h_t(y)}{h_t(x)} u_t(y, x)}$$

Recall Doob's h-transform for a jump process

With Doob's h-transform of a jump process, we tilt every rate by a ratio of h:

$$\boxed{u_t^h(y,x) = \frac{h_t(y)}{h_t(x)}\, u_t(y,x)}$$

The h-transformed rate from x to y equals $h_t(y)$ divided by $h_t(x)$, times the original rate.

Recall Doob's h-transform for a jump process

With Doob's h-transform of a jump process, we tilt every rate by a ratio of h:

$$\boxed{u_t^h(y,x) = \frac{h_t(y)}{h_t(x)}\, u_t(y,x)}$$

The h-transformed rate from x to y equals $h_t(y)$ divided by $h_t(x)$, times the original rate.

When h is harmonic, the transformed process is the original one conditioned on the terminal event.

Recall Doob's h-transform for a jump process

With Doob's h-transform of a jump process, we tilt every rate by a ratio of h:

$$u_t^h(y, x) = \frac{h_t(y)}{h_t(x)} u_t(y, x)$$

The h-transformed rate from x to y equals $h_t(y)$ divided by $h_t(x)$, times the original rate.

When h is harmonic, the transformed process is the original one conditioned on the terminal event.

So the guided sampler is the model conditioned on the property, exact in the continuous-time limit and with the true property model.

Only one position can jump at a time

Only one position can jump at a time

The conditional path factorizes over positions, so two positions never jump at the same instant.

Only one position can jump at a time

The conditional path factorizes over positions, so two positions never jump at the same instant.

From a sequence of D positions over S letters: $\#\{\text{destinations}\} = D\,(S-1)+1$

Only one position can jump at a time

The conditional path factorizes over positions, so two positions never jump at the same instant.

From a sequence of D positions over S letters: $\#\{\text{destinations}\} = D\,(S-1)+1$

The number of states the process can move to is D times S minus one, plus the current state, since exactly one position changes.

Only one position can jump at a time

The conditional path factorizes over positions, so two positions never jump at the same instant.

From a sequence of D positions over S letters: $\#\{\text{destinations}\} = D\,(S-1) + 1$

The number of states the process can move to is D times S minus one, plus the current state, since exactly one position changes.

For a 100-residue peptide over 20 amino acids and a mask token, that is 2001 property evaluations per step, against 21 raised to the 100 states.

Let’s guide one masked base

Let’s guide one masked base

At t = 0.6 with a masked position, the denoiser gives (0.1, 0.2, 0.3, 0.4) over A, C, G, T, so the unguided rates are (0.25, 0.50, 0.75, 1.00).

Let’s guide one masked base

At t = 0.6 with a masked position, the denoiser gives (0.1, 0.2, 0.3, 0.4) over A, C, G, T, so the unguided rates are (0.25, 0.50, 0.75, 1.00).

The property model scores the four unmasked candidates and the masked state: 0.10, 0.35, 0.55, 0.25 against 0.20.

Let’s guide one masked base

At t = 0.6 with a masked position, the denoiser gives (0.1, 0.2, 0.3, 0.4) over A, C, G, T, so the unguided rates are (0.25, 0.50, 0.75, 1.00).

The property model scores the four unmasked candidates and the masked state: 0.10, 0.35, 0.55, 0.25 against 0.20.

$$\text{ratios} = (0.50,\ 1.75,\ 2.75,\ 1.25), \qquad \text{guided rates} = (0.125,\ 0.875,\ 2.063,\ 1.250)$$

Let’s guide one masked base

At t = 0.6 with a masked position, the denoiser gives (0.1, 0.2, 0.3, 0.4) over A, C, G, T, so the unguided rates are (0.25, 0.50, 0.75, 1.00).

The property model scores the four unmasked candidates and the masked state: 0.10, 0.35, 0.55, 0.25 against 0.20.

$$\text{ratios} = (0.50,\ 1.75,\ 2.75,\ 1.25), \qquad \text{guided rates} = (0.125,\ 0.875,\ 2.063,\ 1.250)$$

The likelihood ratios for A, C, G, and T are 0.50, 1.75, 2.75, and 1.25. Multiplying by the unguided rates gives 0.125, 0.875, 2.063, and 1.250.

Let’s guide one masked base

At t = 0.6 with a masked position, the denoiser gives (0.1, 0.2, 0.3, 0.4) over A, C, G, T, so the unguided rates are (0.25, 0.50, 0.75, 1.00).

The property model scores the four unmasked candidates and the masked state: 0.10, 0.35, 0.55, 0.25 against 0.20.

$$\text{ratios} = (0.50,\ 1.75,\ 2.75,\ 1.25), \qquad \text{guided rates} = (0.125,\ 0.875,\ 2.063,\ 1.250)$$

The destination probabilities move from (0.10, 0.20, 0.30, 0.40) to (0.03, 0.20, 0.48, 0.29).

Let’s guide one masked base

At t = 0.6 with a masked position, the denoiser gives (0.1, 0.2, 0.3, 0.4) over A, C, G, T, so the unguided rates are (0.25, 0.50, 0.75, 1.00).

The property model scores the four unmasked candidates and the masked state: 0.10, 0.35, 0.55, 0.25 against 0.20.

$$\text{ratios} = (0.50,\ 1.75,\ 2.75,\ 1.25), \qquad \text{guided rates} = (0.125,\ 0.875,\ 2.063,\ 1.250)$$

The destination probabilities move from (0.10, 0.20, 0.30, 0.40) to (0.03, 0.20, 0.48, 0.29).

The total exit rate rises from 2.50 to 4.31, so the guided chain also unmasks faster.

Raise the likelihood ratio to a power γ

Raise the likelihood ratio to a power γ

$$u_t^{(\gamma)}(y \mid x,\, c) = \left[\frac{p(c \mid y,\, t)}{p(c \mid x,\, t)}\right]^{\gamma} u_t(y, x)$$

Raise the likelihood ratio to a power γ

$$u_t^{(\gamma)}(y \mid x,\, c) = \left[\frac{p(c \mid y,\, t)}{p(c \mid x,\, t)}\right]^{\gamma} u_t(y, x)$$

The guided rate with strength γ raises the likelihood ratio to the power γ before multiplying the unguided rate.

Raise the likelihood ratio to a power γ

$$u_t^{(\gamma)}(y \mid x,\, c) = \left[\frac{p(c \mid y,\, t)}{p(c \mid x,\, t)}\right]^{\gamma} u_t(y, x)$$

The guided rate with strength γ raises the likelihood ratio to the power γ before multiplying the unguided rate.

At γ = 0, we recover the unguided rates. At γ = 1, we recover the exact conditional process.

Raise the likelihood ratio to a power γ

$$u_t^{(\gamma)}(y \mid x,\, c) = \left[\frac{p(c \mid y,\, t)}{p(c \mid x,\, t)}\right]^{\gamma} u_t(y, x)$$

The guided rate with strength γ raises the likelihood ratio to the power γ before multiplying the unguided rate.

At γ = 0, we recover the unguided rates. At γ = 1, we recover the exact conditional process.

At γ = 2 the same example gives destination probabilities (0.01, 0.17, 0.64, 0.18) and an exit rate of 8.83.

Raise the likelihood ratio to a power γ

$$u_t^{(\gamma)}(y \mid x,\, c) = \left[\frac{p(c \mid y,\, t)}{p(c \mid x,\, t)}\right]^{\gamma} u_t(y, x)$$

The guided rate with strength γ raises the likelihood ratio to the power γ before multiplying the unguided rate.

At γ = 0, we recover the unguided rates. At γ = 1, we recover the exact conditional process.

At γ = 2 the same example gives destination probabilities (0.01, 0.17, 0.64, 0.18) and an exit rate of 8.83.

Only γ = 1 is exact because a power of h is generally not harmonic. Larger γ increases the preference for the property and samples a tempered distribution.

Train a conditional model and an unconditional one

Train a conditional model and an unconditional one

After training conditional and unconditional models, combine their rates:

$$\boxed{u_t^{(\gamma)}(y \mid x, c) = u_t(y \mid x, c)^{\gamma}\, u_t(y, x)^{1-\gamma}}$$

Train a conditional model and an unconditional one

After training conditional and unconditional models, combine their rates:

$$\boxed{u_t^{(\gamma)}(y \mid x, c) = u_t(y \mid x, c)^{\gamma}\, u_t(y, x)^{1-\gamma}}$$

The guided rate is the conditional model's rate raised to the power γ, times the unconditional rate raised to the power one minus γ.

Train a conditional model and an unconditional one

After training conditional and unconditional models, combine their rates:

$$\boxed{u_t^{(\gamma)}(y \mid x, c) = u_t(y \mid x, c)^{\gamma}\, u_t(y, x)^{1-\gamma}}$$

The guided rate is the conditional model's rate raised to the power γ, times the unconditional rate raised to the power one minus γ.

In logarithms this is the classifier-free formula from continuous diffusion, written for rates.

Train a conditional model and an unconditional one

After training conditional and unconditional models, combine their rates:

$$\boxed{u_t^{(\gamma)}(y \mid x, c) = u_t(y \mid x, c)^{\gamma}\, u_t(y, x)^{1-\gamma}}$$

The guided rate is the conditional model's rate raised to the power γ, times the unconditional rate raised to the power one minus γ.

In logarithms this is the classifier-free formula from continuous diffusion, written for rates.

The Δt factors cancel between the two expansions, so the product of rates is well defined.

Expand the log ratio to first order

Expand the log ratio to first order

$$\boxed{\log \frac{p_\phi(c \mid y, t)}{p_\phi(c \mid x, t)} \approx (y - x)^\top \nabla_x \log p_\phi(c \mid x, t)}$$

Expand the log ratio to first order

$$\log \frac{p_\phi(c \mid y, t)}{p_\phi(c \mid x, t)} \approx (y - x)^\top \nabla_x \log p_\phi(c \mid x, t)$$

We approximate the log ratio of property probabilities by the inner product of the change in the one-hot sequence with the gradient of the log property probability at the current sequence.

Expand the log ratio to first order

$$\log \frac{p_\phi(c \mid y, t)}{p_\phi(c \mid x, t)} \approx (y - x)^\top \nabla_x \log p_\phi(c \mid x, t)$$

We approximate the log ratio of property probabilities by the inner product of the change in the one-hot sequence with the gradient of the log property probability at the current sequence.

For a single position changing from letter a to letter b, that inner product is the gradient at b minus the gradient at a.

Expand the log ratio to first order

$$\log \frac{p_\phi(c \mid y, t)}{p_\phi(c \mid x, t)} \approx (y - x)^\top \nabla_x \log p_\phi(c \mid x, t)$$

We approximate the log ratio of property probabilities by the inner product of the change in the one-hot sequence with the gradient of the log property probability at the current sequence.

For a single position changing from letter a to letter b, that inner product is the gradient at b minus the gradient at a.

We need one forward pass and one backward pass per step, compared with 1901 forward passes for separate destination scores.

Discrete Guidance: favoring a DNA edit

Toy current base A has property probability 0.2. Changing it to G or C gives property probabilities 0.8 or 0.1. The two base rates are 3 and 2.

Discrete Guidance: favoring a DNA edit

Toy current base A has property probability 0.2. Changing it to G or C gives property probabilities 0.8 or 0.1. The two base rates are 3 and 2.

$$u^c(\mathrm{G}, \mathrm{A}) = 3\frac{0.8}{0.2} = 12, \qquad u^c(\mathrm{C}, \mathrm{A}) = 2\frac{0.1}{0.2} = 1$$

Discrete Guidance: favoring a DNA edit

Toy current base A has property probability 0.2. Changing it to G or C gives property probabilities 0.8 or 0.1. The two base rates are 3 and 2.

$$u^c(\mathrm{G}, \mathrm{A}) = 3\frac{0.8}{0.2} = 12, \qquad u^c(\mathrm{C}, \mathrm{A}) = 2\frac{0.1}{0.2} = 1$$

Conditioning multiplies the G rate by four and halves the C rate.

Discrete Guidance: favoring a DNA edit

Toy current base A has property probability 0.2. Changing it to G or C gives property probabilities 0.8 or 0.1. The two base rates are 3 and 2.

$$u^c(\mathrm{G}, \mathrm{A}) = 3\frac{0.8}{0.2} = 12, \qquad u^c(\mathrm{C}, \mathrm{A}) = 2\frac{0.1}{0.2} = 1$$

Conditioning multiplies the G rate by four and halves the C rate.

$$h = 0.02 : \qquad p_{\mathrm{step}}(\mathrm{G}, \mathrm{C}, \mathrm{A}) = (0.24, 0.02, 0.74)$$

Discrete Guidance: favoring a DNA edit

Toy current base A has property probability 0.2. Changing it to G or C gives property probabilities 0.8 or 0.1. The two base rates are 3 and 2.

$$u^c(\mathrm{G}, \mathrm{A}) = 3\frac{0.8}{0.2} = 12, \qquad u^c(\mathrm{C}, \mathrm{A}) = 2\frac{0.1}{0.2} = 1$$

Conditioning multiplies the G rate by four and halves the C rate.

$$h = 0.02 : \qquad p_{\mathrm{step}}(\mathrm{G}, \mathrm{C}, \mathrm{A}) = (0.24, 0.02, 0.74)$$

In this first-order step, G gets probability 0.24, C gets 0.02, and A gets the remaining 0.74.

Discrete Guidance: favoring a DNA edit

Toy current base A has property probability 0.2. Changing it to G or C gives property probabilities 0.8 or 0.1. The two base rates are 3 and 2.

$$u^c(\mathrm{G}, \mathrm{A}) = 3\frac{0.8}{0.2} = 12, \qquad u^c(\mathrm{C}, \mathrm{A}) = 2\frac{0.1}{0.2} = 1$$

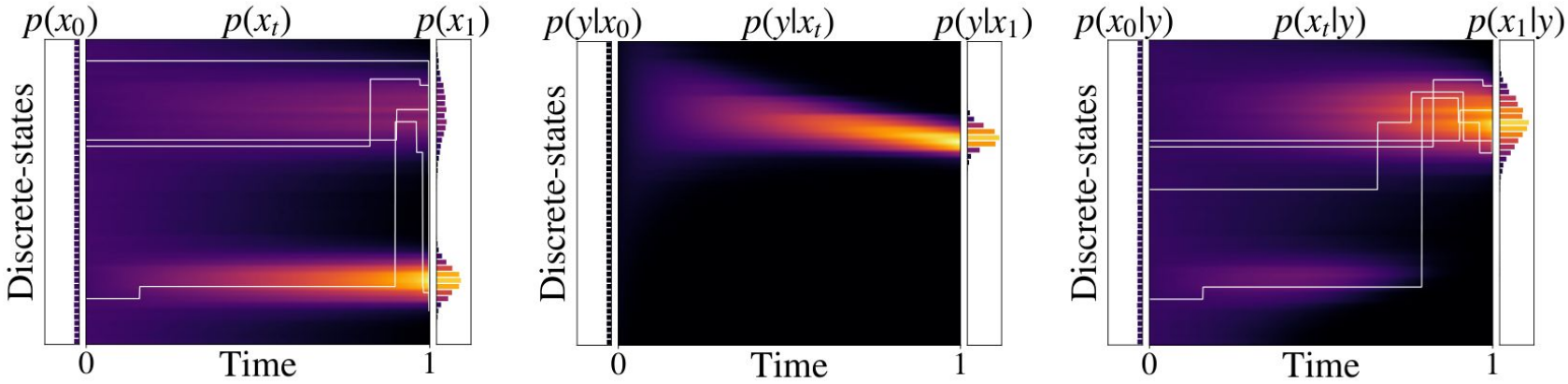
Conditioning multiplies the G rate by four and halves the C rate.

$$h = 0.02 : \qquad p_{\mathrm{step}}(\mathrm{G}, \mathrm{C}, \mathrm{A}) = (0.24, 0.02, 0.74)$$

In this first-order step, G gets probability 0.24, C gets 0.02, and A gets the remaining 0.74.

A move becomes more frequent when it makes the desired property more likely.

With a property model on noisy states, an unconditional sampler becomes a conditional one



The unguided chain, the property model on noisy states, and the guided chain.

Nisonoff,..,Listgarten. *ICLR* (2025)

Guided inverse folding on a hard protein

Method	$P_\theta(\mathbf{x} \mid \mathbf{c})$ Temp	$\Delta\Delta G > 0$ % (↑)	RMSD ≤ 2Å % (↑)	Diversity (↑)	Success (↑)
ProteinMPNN	1.0	0%	47%	53.07	0%
	0.1	1%	100%	27.98	1%
	0.01	0%	100%	27.42	0%
FMIF	1.0	0%	75%	50.29	0%
	0.1	2%	100%	23.13	2%
	0.01	4%	100%	21.82	4%
DiGress ($\gamma = 1$)	0.1	0%	100%	18.65	0%
DiGress ($\gamma = 10$)	0.1	0%	100%	16.76	0%
DiGress ($\gamma = 100$)	0.1	0%	100%	22.54	0%
FMIF-DG-TAG ($\gamma = 1$)	0.1	9%	100%	24.15	9%
FMIF-DG-TAG ($\gamma = 10$)	0.1	52%	100%	23.26	52%
FMIF-DG-TAG ($\gamma = 100$)	0.1	94%	100%	20.82	94%
FMIF-DG-Exact ($\gamma = 1$)	0.1	10%	100%	24.66	10%
FMIF-DG-Exact ($\gamma = 10$)	0.1	58%	100%	25.78	58%
FMIF-DG-Exact ($\gamma = 100$)	0.1	100%	100%	19.72	100%

Nisonoff,..,Listgarten. *ICLR* (2025)

Among ProteinMPNN sequences that fold, 1% are predicted to be stable

Method	$P_\theta(\mathbf{x} \mid \mathbf{c})$ Temp	$\Delta\Delta G > 0$ % (↑)	RMSD ≤ 2Å % (↑)	Diversity (↑)	Success (↑)
ProteinMPNN	1.0	0%	47%	53.07	0%
	0.1	1%	100%	27.98	1%
	0.01	0%	100%	27.42	0%
FMIF	1.0	0%	75%	50.29	0%
	0.1	2%	100%	23.13	2%
	0.01	4%	100%	21.82	4%
DiGress ($\gamma = 1$)	0.1	0%	100%	18.65	0%
DiGress ($\gamma = 10$)	0.1	0%	100%	16.76	0%
DiGress ($\gamma = 100$)	0.1	0%	100%	22.54	0%
FMIF-DG-TAG ($\gamma = 1$)	0.1	9%	100%	24.15	9%
FMIF-DG-TAG ($\gamma = 10$)	0.1	52%	100%	23.26	52%
FMIF-DG-TAG ($\gamma = 100$)	0.1	94%	100%	20.82	94%
FMIF-DG-Exact ($\gamma = 1$)	0.1	10%	100%	24.66	10%
FMIF-DG-Exact ($\gamma = 10$)	0.1	58%	100%	25.78	58%
FMIF-DG-Exact ($\gamma = 100$)	0.1	100%	100%	19.72	100%

Nisonoff,..,Listgarten. *ICLR* (2025)

The unguided flow model reaches 4% at its best temperature

Method	$P_\theta(\mathbf{x} \mid \mathbf{c})$ Temp	$\Delta\Delta G > 0$ % (↑)	RMSD ≤ 2Å % (↑)	Diversity (↑)	Success (↑)
ProteinMPNN	1.0	0%	47%	53.07	0%
	0.1	1%	100%	27.98	1%
	0.01	0%	100%	27.42	0%
FMIF	1.0	0%	75%	50.29	0%
	0.1	2%	100%	23.13	2%
	0.01	4%	100%	21.82	4%
DiGress ($\gamma = 1$)	0.1	0%	100%	18.65	0%
DiGress ($\gamma = 10$)	0.1	0%	100%	16.76	0%
DiGress ($\gamma = 100$)	0.1	0%	100%	22.54	0%
FMIF-DG-TAG ($\gamma = 1$)	0.1	9%	100%	24.15	9%
FMIF-DG-TAG ($\gamma = 10$)	0.1	52%	100%	23.26	52%
FMIF-DG-TAG ($\gamma = 100$)	0.1	94%	100%	20.82	94%
FMIF-DG-Exact ($\gamma = 1$)	0.1	10%	100%	24.66	10%
FMIF-DG-Exact ($\gamma = 10$)	0.1	58%	100%	25.78	58%
FMIF-DG-Exact ($\gamma = 100$)	0.1	100%	100%	19.72	100%

Nisonoff,..,Listgarten. *ICLR* (2025)

The Taylor approximation with γ = 100 reaches 94%

Method	$P_\theta(\mathbf{x} \mid \mathbf{c})$ Temp	$\Delta\Delta G > 0$ % (↑)	RMSD $\leq$ 2Å % (↑)	Diversity (↑)	Success (↑)
ProteinMPNN	1.0	0%	47%	53.07	0%
	0.1	1%	100%	27.98	1%
	0.01	0%	100%	27.42	0%
FMIF	1.0	0%	75%	50.29	0%
	0.1	2%	100%	23.13	2%
	0.01	4%	100%	21.82	4%
DiGress ($\gamma = 1$)	0.1	0%	100%	18.65	0%
DiGress ($\gamma = 10$)	0.1	0%	100%	16.76	0%
DiGress ($\gamma = 100$)	0.1	0%	100%	22.54	0%
FMIF-DG-TAG ($\gamma = 1$)	0.1	9%	100%	24.15	9%
FMIF-DG-TAG ($\gamma = 10$)	0.1	52%	100%	23.26	52%
FMIF-DG-TAG ($\gamma = 100$)	0.1	94%	100%	20.82	94%
FMIF-DG-Exact ($\gamma = 1$)	0.1	10%	100%	24.66	10%
FMIF-DG-Exact ($\gamma = 10$)	0.1	58%	100%	25.78	58%
FMIF-DG-Exact ($\gamma = 100$)	0.1	100%	100%	19.72	100%

Nisonoff,..,Listgarten. *ICLR* (2025)

Exact guidance reaches 100%, with every sequence folding to within 2 angstroms

Method	$P_\theta(\mathbf{x} \mid \mathbf{c})$ Temp	$\Delta\Delta G > 0$ % (↑)	RMSD ≤ 2Å % (↑)	Diversity (↑)	Success (↑)
ProteinMPNN	1.0	0%	47%	53.07	0%
	0.1	1%	100%	27.98	1%
	0.01	0%	100%	27.42	0%
FMIF	1.0	0%	75%	50.29	0%
	0.1	2%	100%	23.13	2%
	0.01	4%	100%	21.82	4%
DiGress ($\gamma = 1$)	0.1	0%	100%	18.65	0%
DiGress ($\gamma = 10$)	0.1	0%	100%	16.76	0%
DiGress ($\gamma = 100$)	0.1	0%	100%	22.54	0%
FMIF-DG-TAG ($\gamma = 1$)	0.1	9%	100%	24.15	9%
FMIF-DG-TAG ($\gamma = 10$)	0.1	52%	100%	23.26	52%
FMIF-DG-TAG ($\gamma = 100$)	0.1	94%	100%	20.82	94%
FMIF-DG-Exact ($\gamma = 1$)	0.1	10%	100%	24.66	10%
FMIF-DG-Exact ($\gamma = 10$)	0.1	58%	100%	25.78	58%
FMIF-DG-Exact ($\gamma = 100$)	0.1	100%	100%	19.72	100%

Nisonoff,..,Listgarten. *ICLR* (2025)

On another protein the same γ = 100 collapses to 42%, so the strength is a hyperparameter

Method	$P_\theta(\mathbf{x} \mid \mathbf{c})$ Temp	$\Delta\Delta G > 0$ % (↑)	RMSD $\leq$ 2Å % (↑)	Diversity (↑)	Success (↑)
ProteinMPNN	1.0	0%	47%	53.07	0%
	0.1	1%	100%	27.98	1%
	0.01	0%	100%	27.42	0%
FMIF	1.0	0%	75%	50.29	0%
	0.1	2%	100%	23.13	2%
	0.01	4%	100%	21.82	4%
DiGress ($\gamma = 1$)	0.1	0%	100%	18.65	0%
DiGress ($\gamma = 10$)	0.1	0%	100%	16.76	0%
DiGress ($\gamma = 100$)	0.1	0%	100%	22.54	0%
FMIF-DG-TAG ($\gamma = 1$)	0.1	9%	100%	24.15	9%
FMIF-DG-TAG ($\gamma = 10$)	0.1	52%	100%	23.26	52%
FMIF-DG-TAG ($\gamma = 100$)	0.1	94%	100%	20.82	94%
FMIF-DG-Exact ($\gamma = 1$)	0.1	10%	100%	24.66	10%
FMIF-DG-Exact ($\gamma = 10$)	0.1	58%	100%	25.78	58%
FMIF-DG-Exact ($\gamma = 100$)	0.1	100%	100%	19.72	100%

Nisonoff,..,Listgarten. *ICLR* (2025)

What if we need to improve several peptide properties at once?

MOG-DFM: Multi-Objective-Guided Discrete Flow Matching

Multi-Objective-Guided Discrete Flow Matching for Controllable Biological Sequence Design

Tong Chen[1,2], **Yinuo Zhang**[1,3], **Sophia Tang**[1,4], **Pranam Chatterjee**[1,5,6,†]

[1]Department of Biomedical Engineering, Duke University
[2]Department of Computer Science, Fudan University
[3]Center of Computational Biology, Duke-NUS Medical School
[4]Management and Technology Program, University of Pennsylvania
[5]Department of Computer Science, Duke University
[6]Department of Biostatistics and Bioinformatics, Duke University

[†]Corresponding author: pranam.chatterjee@duke.edu

Chen,..,Chatterjee. *ICML Workshop* (2025)

Peptide design has five objectives at once

Peptide design has five objectives at once

They are binding affinity, solubility, non-fouling, hemolysis, and half-life, each from a separate property model.

Peptide design has five objectives at once

They are binding affinity, solubility, non-fouling, hemolysis, and half-life, each from a separate property model.

We lack the joint property model needed for one likelihood ratio per objective.

Peptide design has five objectives at once

They are binding affinity, solubility, non-fouling, hemolysis, and half-life, each from a separate property model.

We lack the joint property model needed for one likelihood ratio per objective.

The goal is the Pareto front:

$$\mathrm{PF} = \{\, x \;:\; \nexists\, x' \text{ with } s_n(x') \geq s_n(x)\ \forall n \text{ and } s_m(x') > s_m(x) \text{ for some } m \,\}$$

Peptide design has five objectives at once

They are binding affinity, solubility, non-fouling, hemolysis, and half-life, each from a separate property model.

We lack the joint property model needed for one likelihood ratio per objective.

The goal is the Pareto front:

$$\mathrm{PF} = \{\, x \;:\; \nexists\, x' \text{ with } s_n(x') \geq s_n(x)\ \forall n \text{ and } s_m(x') > s_m(x) \text{ for some } m \,\}$$

The Pareto front is the set of sequences for which no other sequence is at least as good on every objective and strictly better on one.

Peptide design has five objectives at once

They are binding affinity, solubility, non-fouling, hemolysis, and half-life, each from a separate property model.

We lack the joint property model needed for one likelihood ratio per objective.

The goal is the Pareto front:

$$\mathrm{PF} = \{\, x \;:\; \nexists\, x' \text{ with } s_n(x') \geq s_n(x)\ \forall n \text{ and } s_m(x') > s_m(x) \text{ for some } m \,\}$$

The Pareto front is the set of sequences for which no other sequence is at least as good on every objective and strictly better on one.

In MOG-DFM, we steer toward one point on that front, chosen by a weight vector ω drawn from a simplex lattice.

In MOG-DFM, we score every candidate move twice

In MOG-DFM, we score every candidate move twice

First by rank, within objective n: $$I_n(y^i, x) = \frac{\mathrm{rank}\big(s_n(x^{(i \to y^i)}) - s_n(x)\big)}{|\mathcal{T}|}$$

In MOG-DFM, we score every candidate move twice

First by rank, within objective n: $$I_n(y^i, x) = \frac{\mathrm{rank}\big(s_n(x^{(i \to y^i)}) - s_n(x)\big)}{|\mathcal{T}|}$$

The rank score of a candidate letter is the rank of its score change on objective n among all candidate letters at that position, divided by the alphabet size.

In MOG-DFM, we score every candidate move twice

First by rank, within objective n: $$I_n(y^i, x) = \frac{\mathrm{rank}\big(s_n(x^{(i \to y^i)}) - s_n(x)\big)}{|\mathcal{T}|}$$

The rank score of a candidate letter is the rank of its score change on objective n among all candidate letters at that position, divided by the alphabet size.

Second by direction, along the chosen trade-off: $$D(y^i, x, \omega) = \Delta \mathbf{s}(y^i, x) \cdot \omega$$

In MOG-DFM, we score every candidate move twice

First by rank, within objective n: $$I_n(y^i, x) = \frac{\mathrm{rank}\big(s_n(x^{(i \to y^i)}) - s_n(x)\big)}{|\mathcal{T}|}$$

The rank score of a candidate letter is the rank of its score change on objective n among all candidate letters at that position, divided by the alphabet size.

Second by direction, along the chosen trade-off: $$D(y^i, x, \omega) = \Delta \mathbf{s}(y^i, x) \cdot \omega$$

The directional score is the inner product of the vector of score changes with the weight vector ω.

In MOG-DFM, we score every candidate move twice

First by rank, within objective n: $$I_n(y^i, x) = \frac{\mathrm{rank}\big(s_n(x^{(i \to y^i)}) - s_n(x)\big)}{|\mathcal{T}|}$$

The rank score of a candidate letter is the rank of its score change on objective n among all candidate letters at that position, divided by the alphabet size.

Second by direction, along the chosen trade-off: $$D(y^i, x, \omega) = \Delta \mathbf{s}(y^i, x) \cdot \omega$$

The directional score is the inner product of the vector of score changes with the weight vector ω.

We use ranks to compare objectives with different units. The inner product measures progress along the chosen direction.

The two scores become an exponential tilt

The two scores become an exponential tilt

$$\Delta S(y^i, x, \omega) = \mathrm{Norm}\left[\frac{1}{N}\sum_n i_n I_n(y^i, x)\right] + \lambda\,\mathrm{Norm}\left[D(y^i, x, \omega)\right]$$

The two scores become an exponential tilt

$$\Delta S(y^i, x, \omega) = \mathrm{Norm}\left[\frac{1}{N}\sum_n i_n I_n(y^i, x)\right] + \lambda\,\mathrm{Norm}\left[D(y^i, x, \omega)\right]$$

The combined score is the normalized importance-weighted average of the rank scores, plus λ times the normalized directional score, where Norm means z-scoring across the candidate letters.

The two scores become an exponential tilt

$$\Delta S(y^i, x, \omega) = \mathrm{Norm}\Big[\frac{1}{N}\sum_n i_n I_n(y^i, x)\Big] + \lambda\,\mathrm{Norm}\big[D(y^i, x, \omega)\big]$$

The combined score is the normalized importance-weighted average of the rank scores, plus λ times the normalized directional score, where Norm means z-scoring across the candidate letters.

$$u^i_{\mathrm{guided},t}(y^i, x \mid \omega) = \beta\, u^i_t(y^i, x)\, \exp\big(\Delta S(y^i, x, \omega)\big)$$

The two scores become an exponential tilt

$$\Delta S(y^i, x, \omega) = \mathrm{Norm}\Big[\frac{1}{N}\sum_n i_n I_n(y^i, x)\Big] + \lambda\,\mathrm{Norm}\big[D(y^i, x, \omega)\big]$$

The combined score is the normalized importance-weighted average of the rank scores, plus λ times the normalized directional score, where Norm means z-scoring across the candidate letters.

$$u^i_{\mathrm{guided},t}(y^i, x \mid \omega) = \beta\, u^i_t(y^i, x)\, \exp\big(\Delta S(y^i, x, \omega)\big)$$

The guided rate is β times the pretrained rate, times the exponential of the combined score.

The two scores become an exponential tilt

$$\Delta S(y^i, x, \omega) = \mathrm{Norm}\Big[\frac{1}{N}\sum_n i_n I_n(y^i, x)\Big] + \lambda\,\mathrm{Norm}\big[D(y^i, x, \omega)\big]$$

The combined score is the normalized importance-weighted average of the rank scores, plus λ times the normalized directional score, where Norm means z-scoring across the candidate letters.

$$u^i_{\mathrm{guided},t}(y^i, x \mid \omega) = \beta\, u^i_t(y^i, x)\, \exp\big(\Delta S(y^i, x, \omega)\big)$$

The guided rate is β times the pretrained rate, times the exponential of the combined score.

The exponential factor is positive. After setting the diagonal to minus the outgoing rate sum, we have a valid velocity.

The two scores become an exponential tilt

$$\Delta S(y^i, x, \omega) = \mathrm{Norm}\Big[\frac{1}{N}\sum_n i_n I_n(y^i, x)\Big] + \lambda\,\mathrm{Norm}\big[D(y^i, x, \omega)\big]$$

The combined score is the normalized importance-weighted average of the rank scores, plus λ times the normalized directional score, where Norm means z-scoring across the candidate letters.

$$u^i_{\mathrm{guided},t}(y^i, x \mid \omega) = \beta\, u^i_t(y^i, x)\, \exp\big(\Delta S(y^i, x, \omega)\big)$$

The guided rate is β times the pretrained rate, times the exponential of the combined score.

The exponential factor is positive. After setting the diagonal to minus the outgoing rate sum, we have a valid velocity.

We use a heuristic tilt here. Without an h(y)/h(x) ratio, we cannot condition exactly.

The two scores become an exponential tilt

$$\Delta S(y^i, x, \omega) = \mathrm{Norm}\Big[\frac{1}{N}\sum_n i_n I_n(y^i, x)\Big] + \lambda\,\mathrm{Norm}\big[D(y^i, x, \omega)\big]$$

The combined score is the normalized importance-weighted average of the rank scores, plus λ times the normalized directional score, where Norm means z-scoring across the candidate letters.

$$u^i_{\mathrm{guided},t}(y^i, x \mid \omega) = \beta\, u^i_t(y^i, x)\, \exp\big(\Delta S(y^i, x, \omega)\big)$$

The guided rate is β times the pretrained rate, times the exponential of the combined score.

The exponential factor is positive. After setting the diagonal to minus the outgoing rate sum, we have a valid velocity.

A good score speeds up a move only if the base model already permits it.

Accept a candidate only if its improvement sits inside a cone

Accept a candidate only if its improvement sits inside a cone

Accept a candidate only if its improvement sits inside a cone around ω:

$$\boxed{\alpha^i = \arccos\left(\frac{\Delta\mathbf{s}(y^i, x) \cdot \omega}{\|\Delta\mathbf{s}(y^i, x)\| \, \|\omega\|}\right) \leq \Phi}$$

Accept a candidate only if its improvement sits inside a cone

Accept a candidate only if its improvement sits inside a cone around ω:

$$\boxed{\alpha^i = \arccos\left(\frac{\Delta\mathbf{s}(y^i, x) \cdot \omega}{\|\Delta\mathbf{s}(y^i, x)\| \, \|\omega\|}\right) \leq \Phi}$$

The alignment angle is the arccosine of the normalized inner product between the score change and ω. We accept the candidate when that angle is at most Φ.

Accept a candidate only if its improvement sits inside a cone

Accept a candidate only if its improvement sits inside a cone around ω:

$$\boxed{\alpha^i = \arccos\left(\frac{\Delta \mathbf{s}(y^i, x) \cdot \omega}{\|\Delta \mathbf{s}(y^i, x)\| \, \|\omega\|}\right) \leq \Phi}$$

The alignment angle is the arccosine of the normalized inner product between the score change and ω. We accept the candidate when that angle is at most Φ.

Inside the cone, accepted moves improve the weighted objective in expectation because their directional scores are positive. The theorem requires Φ to be less than a right angle.

Update the cone angle after each step

Update the cone angle after each step

$$\Phi_{t+h} = \mathrm{clip}\Big(\Phi_t \exp\big(\eta\,(\bar{r}_t - \tau)\big),\ \Phi_{\min},\ \Phi_{\max}\Big)$$

Update the cone angle after each step

$$\Phi_{t+h} = \mathrm{clip}\Big(\Phi_t \exp\big(\eta\,(\bar{r}_t - \tau)\big),\ \Phi_{\min},\ \Phi_{\max}\Big)$$

The next cone angle is the current angle times the exponential of η times the smoothed rejection rate minus its target, clipped to a range.

Update the cone angle after each step

$$\Phi_{t+h} = \mathrm{clip}\Big(\Phi_t \exp\left(\eta\left(\bar{r}_t - \tau\right)\right),\ \Phi_{\min},\ \Phi_{\max}\Big)$$

The next cone angle is the current angle times the exponential of η times the smoothed rejection rate minus its target, clipped to a range.

We widen the cone when the rejection rate is too high and narrow it when the rate is too low. This is proportional control of log Φ.

Update the cone angle after each step

$$\Phi_{t+h} = \operatorname{clip}\Big(\Phi_t \exp\big(\eta\,(\bar{r}_t - \tau)\big),\ \Phi_{\min},\ \Phi_{\max}\Big)$$

The next cone angle is the current angle times the exponential of η times the smoothed rejection rate minus its target, clipped to a range.

We widen the cone when the rejection rate is too high and narrow it when the rate is too low. This is proportional control of log Φ.

Removing the cone costs most of the half-life gain in their ablation, from about 30 hours down to about 12.

MOG-DFM: which improvement fits the trade-off?

Toy improvements on two equally weighted objectives are (2, 1) for edit G and (–1, 2) for edit C. The chosen direction is (0.5, 0.5), with a 30-degree acceptance cone.

MOG-DFM: which improvement fits the trade-off?

Toy improvements on two equally weighted objectives are (2, 1) for edit G and (–1, 2) for edit C. The chosen direction is (0.5, 0.5), with a 30-degree acceptance cone.

$$D_{\mathrm{G}} = 0.5(2) + 0.5(1) = 1.5, \qquad D_{\mathrm{C}} = 0.5(-1) + 0.5(2) = 0.5$$

MOG-DFM: which improvement fits the trade-off?

Toy improvements on two equally weighted objectives are (2, 1) for edit G and (–1, 2) for edit C. The chosen direction is (0.5, 0.5), with a 30-degree acceptance cone.

$$D_{\mathrm{G}} = 0.5(2) + 0.5(1) = 1.5, \qquad D_{\mathrm{C}} = 0.5(-1) + 0.5(2) = 0.5$$

Both edits increase the weighted score, although C worsens the first objective.

MOG-DFM: which improvement fits the trade-off?

Toy improvements on two equally weighted objectives are (2, 1) for edit G and (−1, 2) for edit C. The chosen direction is (0.5, 0.5), with a 30-degree acceptance cone.

$$D_{\mathrm{G}} = 0.5(2) + 0.5(1) = 1.5, \qquad D_{\mathrm{C}} = 0.5(-1) + 0.5(2) = 0.5$$

Both edits increase the weighted score, although C worsens the first objective.

$$\alpha_{\mathrm{G}} = \arccos \frac{3}{\sqrt{10}} \approx 18.4^{\circ}, \qquad \alpha_{\mathrm{C}} = \arccos \frac{1}{\sqrt{10}} \approx 71.6^{\circ}$$

MOG-DFM: which improvement fits the trade-off?

Toy improvements on two equally weighted objectives are (2, 1) for edit G and (−1, 2) for edit C. The chosen direction is (0.5, 0.5), with a 30-degree acceptance cone.

$$D_{\mathrm{G}} = 0.5(2) + 0.5(1) = 1.5, \qquad D_{\mathrm{C}} = 0.5(-1) + 0.5(2) = 0.5$$

Both edits increase the weighted score, although C worsens the first objective.

$$\alpha_{\mathrm{G}} = \arccos \frac{3}{\sqrt{10}} \approx 18.4^{\circ}, \qquad \alpha_{\mathrm{C}} = \arccos \frac{1}{\sqrt{10}} \approx 71.6^{\circ}$$

G is inside the 30-degree cone. We reject C because it is outside.

MOG-DFM: which improvement fits the trade-off?

Toy improvements on two equally weighted objectives are (2, 1) for edit G and (–1, 2) for edit C. The chosen direction is (0.5, 0.5), with a 30-degree acceptance cone.

$$D_{\mathrm{G}} = 0.5(2) + 0.5(1) = 1.5, \qquad D_{\mathrm{C}} = 0.5(-1) + 0.5(2) = 0.5$$

Both edits increase the weighted score, although C worsens the first objective.

$$\alpha_{\mathrm{G}} = \arccos \frac{3}{\sqrt{10}} \approx 18.4^{\circ}, \qquad \alpha_{\mathrm{C}} = \arccos \frac{1}{\sqrt{10}} \approx 71.6^{\circ}$$

G is inside the 30-degree cone. We reject C because it is outside.

We use the cone to retain improvements that follow the requested balance between objectives.

Score every candidate, keep the ones inside the cone, and take one jump

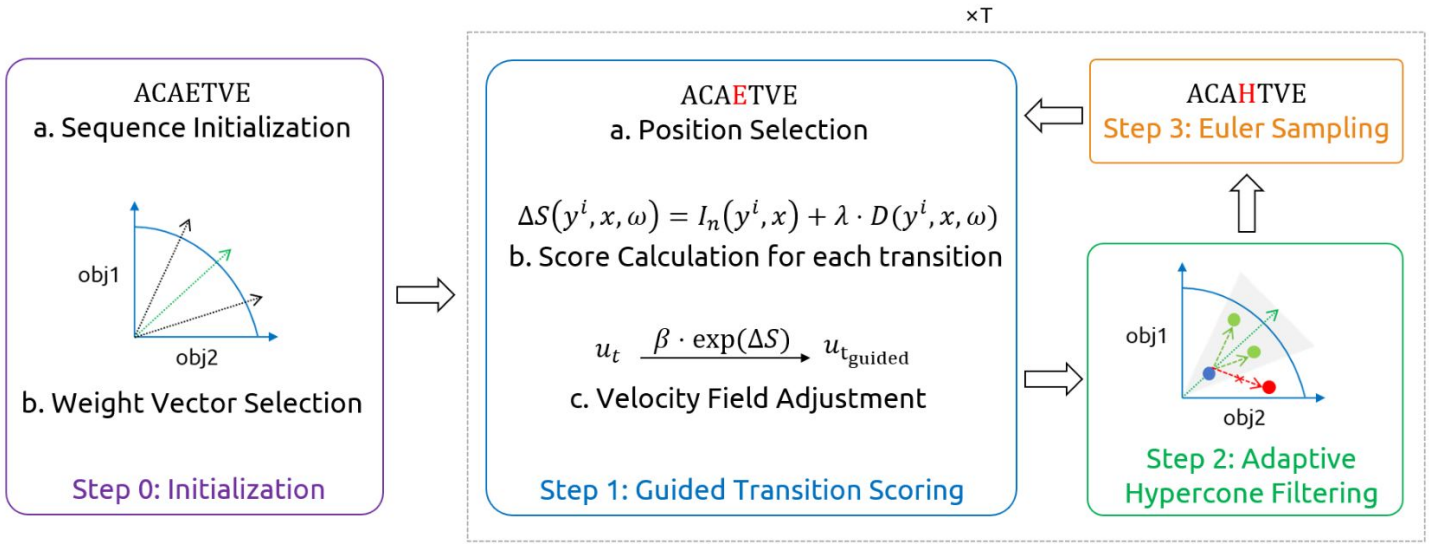


Figure 1: Visualization for MOG-DFM algorithm.

We draw one weight vector, then repeat scoring, filtering, and sampling.

Chen,..,Chatterjee. *ICML Workshop* (2025)

Let’s compare with classical multi-objective optimizers

Target	Method	Time (s)	Hemolysis (↓)	Non-Fouling	Solubility	Half-Life	Affinity
1B8Q	MOPSO	8.54	0.1066	0.4763	0.4684	4.449	6.0594
	NSGA-III	33.13	0.0862	0.5715	0.5825	7.324	7.2178
	SMS-EMOA	**8.21**	0.1196	0.3450	0.3511	3.023	5.955
	SPEA2	17.48	0.0819	0.4973	0.5057	4.126	**7.324**
	MOG-DFM	43.00	**0.0785**	**0.8445**	**0.8455**	**27.227**	5.9094
PPP5	MOPSO	11.34	0.0883	0.4711	0.4255	1.769	6.6958
	NSGA-III	37.30	**0.0479**	0.7138	0.7066	2.901	7.3789
	SMS-EMOA	**8.43**	0.1242	0.4269	0.4334	1.031	6.2854
	SPEA2	19.02	0.0555	0.6221	0.6098	2.613	**7.6253**
	MOG-DFM	90.00	0.0617	**0.7738**	**0.751**	**27.775**	6.8197

Chen,..,Chatterjee. *ICML Workshop* (2025)

A genetic algorithm reaches non-fouling 0.57 and a half-life of 7.3 hours on this target

Target	Method	Time (s)	Hemolysis (↓)	Non-Fouling	Solubility	Half-Life	Affinity
	MOPSO	8.54	0.1066	0.4763	0.4684	4.449	6.0594
	NSGA-III	33.13	0.0862	0.5715	0.5825	7.324	7.2178
1B8Q	SMS-EMOA	**8.21**	0.1196	0.3450	0.3511	3.023	5.955
	SPEA2	17.48	0.0819	0.4973	0.5057	4.126	**7.324**
	MOG-DFM	43.00	**0.0785**	**0.8445**	**0.8455**	**27.227**	5.9094
	MOPSO	11.34	0.0883	0.4711	0.4255	1.769	6.6958
	NSGA-III	37.30	**0.0479**	0.7138	0.7066	2.901	7.3789
PPP5	SMS-EMOA	**8.43**	0.1242	0.4269	0.4334	1.031	6.2854
	SPEA2	19.02	0.0555	0.6221	0.6098	2.613	**7.6253**
	MOG-DFM	90.00	0.0617	**0.7738**	**0.751**	**27.775**	6.8197

Chen,..,Chatterjee. *ICML Workshop* (2025)

MOG-DFM reaches 0.84 and 27.2 hours, with hemolysis also lower

Target	Method	Time (s)	Hemolysis (↓)	Non-Fouling	Solubility	Half-Life	Affinity
1B8Q	MOPSO	8.54	0.1066	0.4763	0.4684	4.449	6.0594
	NSGA-III	33.13	0.0862	0.5715	0.5825	7.324	7.2178
	SMS-EMOA	**8.21**	0.1196	0.3450	0.3511	3.023	5.955
	SPEA2	17.48	0.0819	0.4973	0.5057	4.126	**7.324**
	MOG-DFM	43.00	**0.0785**	**0.8445**	**0.8455**	**27.227**	5.9094
PPP5	MOPSO	11.34	0.0883	0.4711	0.4255	1.769	6.6958
	NSGA-III	37.30	**0.0479**	0.7138	0.7066	2.901	7.3789
	SMS-EMOA	**8.43**	0.1242	0.4269	0.4334	1.031	6.2854
	SPEA2	19.02	0.0555	0.6221	0.6098	2.613	**7.6253**
	MOG-DFM	90.00	0.0617	**0.7738**	**0.751**	**27.775**	6.8197

Chen,..,Chatterjee. *ICML Workshop* (2025)

Binding affinity is 5.91, compared with 7.32 for the best baseline, showing the trade-off with the other properties

Target	Method	Time (s)	Hemolysis (↓)	Non-Fouling	Solubility	Half-Life	Affinity
1B8Q	MOPSO	8.54	0.1066	0.4763	0.4684	4.449	6.0594
	NSGA-III	33.13	0.0862	0.5715	0.5825	7.324	7.2178
	SMS-EMOA	**8.21**	0.1196	0.3450	0.3511	3.023	5.955
	SPEA2	17.48	0.0819	0.4973	0.5057	4.126	**7.324**
	MOG-DFM	43.00	**0.0785**	**0.8445**	**0.8455**	**27.227**	5.9094
PPP5	MOPSO	11.34	0.0883	0.4711	0.4255	1.769	6.6958
	NSGA-III	37.30	**0.0479**	0.7138	0.7066	2.901	7.3789
	SMS-EMOA	**8.43**	0.1242	0.4269	0.4334	1.031	6.2854
	SPEA2	19.02	0.0555	0.6221	0.6098	2.613	**7.6253**
	MOG-DFM	90.00	0.0617	**0.7738**	**0.751**	**27.775**	6.8197

Chen,..,Chatterjee. *ICML Workshop* (2025)

We also improve non-fouling, solubility, and half-life on the second target, using two to ten times the baseline runtime

Target	Method	Time (s)	Hemolysis (↓)	Non-Fouling	Solubility	Half-Life	Affinity
1B8Q	MOPSO	8.54	0.1066	0.4763	0.4684	4.449	6.0594
	NSGA-III	33.13	0.0862	0.5715	0.5825	7.324	7.2178
	SMS-EMOA	**8.21**	0.1196	0.3450	0.3511	3.023	5.955
	SPEA2	17.48	0.0819	0.4973	0.5057	4.126	**7.324**
	MOG-DFM	43.00	**0.0785**	**0.8445**	**0.8455**	**27.227**	5.9094
PPP5	MOPSO	11.34	0.0883	0.4711	0.4255	1.769	6.6958
	NSGA-III	37.30	**0.0479**	0.7138	0.7066	2.901	7.3789
	SMS-EMOA	**8.43**	0.1242	0.4269	0.4334	1.031	6.2854
	SPEA2	19.02	0.0555	0.6221	0.6098	2.613	**7.6253**
	MOG-DFM	90.00	0.0617	**0.7738**	**0.751**	**27.775**	6.8197

Chen,..,Chatterjee. *ICML Workshop* (2025)

Could we guide insertions and deletions while also enforcing constraints on the finished sequence?

pCoMole: Pareto-Constrained Molecule Editing

pCoMole: Pareto-Constrained Molecule Editing with Discrete Flows

Tong Chen,[1,*] **Maximilian Holsman,**[2,*] **Lin Zhao,**[3,*] **Pranam Chatterjee**[1,3,†]

[1]Department of Computer and Information Science, University of Pennsylvania
[2]Department of Computer Science, Duke University
[3]Department of Bioengineering, University of Pennsylvania

[*]Equal contribution
[†]**Corresponding author:** pranam@upenn.edu
Code: https://huggingface.co/ChatterjeeLab/pCoMole

Chen,..,Chatterjee. *NeurIPS* (2026)

Take the same question to Edit Flows

Take the same question to Edit Flows

Edit Flows change the length, so the same multi-objective question arises for sequences that grow and shrink.

Take the same question to Edit Flows

Edit Flows change the length, so the same multi-objective question arises for sequences that grow and shrink.

In pCoMole, we keep the Doob h-transform and build the tilt from a utility in place of a likelihood.

Take the same question to Edit Flows

Edit Flows change the length, so the same multi-objective question arises for sequences that grow and shrink.

In pCoMole, we keep the Doob h-transform and build the tilt from a utility in place of a likelihood.

Score a sequence by its worst objective, plus a small sum:

$$U(x) = \min_k \omega_k\big(f_k(x) - r_k\big) \;+\; \rho \sum_k \omega_k\big(f_k(x) - r_k\big)$$

Take the same question to Edit Flows

Edit Flows change the length, so the same multi-objective question arises for sequences that grow and shrink.

In pCoMole, we keep the Doob h-transform and build the tilt from a utility in place of a likelihood.

Score a sequence by its worst objective, plus a small sum:

$$U(x) = \min_k \omega_k\big(f_k(x) - r_k\big) \;+\; \rho \sum_k \omega_k\big(f_k(x) - r_k\big)$$

The utility of a sequence is the smallest weighted gap between an objective value $f_k(x)$ and its reference level r_k, plus ρ times the sum of those weighted gaps.

Take the same question to Edit Flows

Edit Flows change the length, so the same multi-objective question arises for sequences that grow and shrink.

In pCoMole, we keep the Doob h-transform and build the tilt from a utility in place of a likelihood.

Score a sequence by its worst objective, plus a small sum:

$$U(x) = \min_k \omega_k\big(f_k(x) - r_k\big) \;+\; \rho \sum_k \omega_k\big(f_k(x) - r_k\big)$$

The utility of a sequence is the smallest weighted gap between an objective value $f_k(x)$ and its reference level r_k, plus ρ times the sum of those weighted gaps.

The first term rewards the weakest objective, so a sequence cannot trade solubility for affinity. The second term breaks ties.

Turn that utility into a terminal reward

Turn that utility into a terminal reward

$$\boxed{G(x) = \exp\big(\beta\, U(x)\big) \mathbb{1}\!\!\!

Turn that utility into a terminal reward

$$\boxed{G(x) = \exp\big(\beta\, U(x)\big) \mathbb{1}\big[x \in \mathcal{F}\big]}$$

The terminal reward is the exponential of β times the utility, multiplied by the indicator that the sequence is feasible.

Turn that utility into a terminal reward

$$\boxed{G(x) = \exp\left(\beta\, U(x)\right) \mathbb{1}\left[x \in \mathcal{F}\right]}$$

The terminal reward is the exponential of β times the utility, multiplied by the indicator that the sequence is feasible.

We assign zero reward to infeasible sequences, regardless of their other scores.

Turn that utility into a terminal reward

$$\boxed{G(x) = \exp\big(\beta\, U(x)\big) \mathbb{1}\big[x \in \mathcal{F}\big]}$$

The terminal reward is the exponential of β times the utility, multiplied by the indicator that the sequence is feasible.

We assign zero reward to infeasible sequences, regardless of their other scores.

With the inverse temperature β, we set how strongly high utility is weighted.

Turn that utility into a terminal reward

$$\boxed{G(x) = \exp\big(\beta\, U(x)\big) \mathbb{1}\big[x \in \mathcal{F}\big]}$$

The terminal reward is the exponential of β times the utility, multiplied by the indicator that the sequence is feasible.

We assign zero reward to infeasible sequences, regardless of their other scores.

A high property score only matters after the finished sequence passes every feasibility check.

Put that reward through the same h-transform

Put that reward through the same h-transform

$$h_t(x) = \mathbb{E}\big[\, G(X_1) \,\big|\, X_t = x \,\big], \qquad u_t^G(y \mid x) = u_t(y \mid x)\, \frac{h_t(y)}{h_t(x)}$$

Put that reward through the same h-transform

$$h_t(x) = \mathbb{E}\big[\, G(X_1) \,\big|\, X_t = x \,\big], \qquad u_t^G(y \mid x) = u_t(y \mid x)\, \frac{h_t(y)}{h_t(x)}$$

The function $h_t(x)$ is the expected terminal reward given the current state. We multiply each base rate by the ratio of h at the destination to h at the current state.

Put that reward through the same h-transform

$$h_t(x) = \mathbb{E}\big[\, G(X_1) \,\big|\, X_t = x \,\big], \qquad u_t^G(y \mid x) = u_t(y \mid x)\, \frac{h_t(y)}{h_t(x)}$$

The function $h_t(x)$ is the expected terminal reward given the current state. We multiply each base rate by the ratio of h at the destination to h at the current state.

Their Proposition 3.1 shows the tilted process ends at the base distribution reweighted by G.

Put that reward through the same h-transform

$$h_t(x) = \mathbb{E}\big[\, G(X_1) \,\big|\, X_t = x \,\big], \qquad u_t^G(y \mid x) = u_t(y \mid x)\, \frac{h_t(y)}{h_t(x)}$$

The function $h_t(x)$ is the expected terminal reward given the current state. We multiply each base rate by the ratio of h at the destination to h at the current state.

Their Proposition 3.1 shows the tilted process ends at the base distribution reweighted by G.

We recover the Discrete Guidance construction with a reward in place of an event. Along the path, h is a martingale whose terminal value reweights the endpoint distribution.

We estimate that expectation by rolling the flow forward

We estimate that expectation by rolling the flow forward

From each candidate, run R short continuations to t = 1:

$$\hat{h}_t(y_c) = \frac{1}{R} \sum_{r=1}^{R} G\left(X_1^{(c,r)}\right)$$

We estimate that expectation by rolling the flow forward

From each candidate, run R short continuations to t = 1:

$$\hat{h}_t(y_c) = \frac{1}{R} \sum_{r=1}^{R} G\left(X_1^{(c,r)}\right)$$

The estimate of h at a candidate is the average terminal reward over R rollouts started from that candidate and run to the end of the path.

We estimate that expectation by rolling the flow forward

From each candidate, run R short continuations to t = 1:

$$\hat{h}_t(y_c) = \frac{1}{R} \sum_{r=1}^{R} G\left(X_1^{(c,r)}\right)$$

The estimate of h at a candidate is the average terminal reward over R rollouts started from that candidate and run to the end of the path.

We generate rollouts with the base Edit Flow, so we do not need to retrain it.

We estimate that expectation by rolling the flow forward

From each candidate, run R short continuations to t = 1:

$$\hat{h}_t(y_c) = \frac{1}{R} \sum_{r=1}^{R} G\left(X_1^{(c,r)}\right)$$

The estimate of h at a candidate is the average terminal reward over R rollouts started from that candidate and run to the end of the path.

We generate rollouts with the base Edit Flow, so we do not need to retrain it.

We assign zero value to candidates whose rollouts are all infeasible. If every candidate has zero value, we keep the current sequence.

Choose one of the candidates at each step

Choose one of the candidates at each step

$$\boxed{\tilde{u}_t(y_c \mid x_t) \;\propto\; u_t(y_c \mid x_t)\,\hat{h}_t(y_c), \qquad c = 1, \ldots, C}$$

Choose one of the candidates at each step

$$\tilde{u}_t(y_c \mid x_t) \;\propto\; u_t(y_c \mid x_t)\,\hat{h}_t(y_c), \qquad c = 1, \ldots, C$$

The selection probability of candidate c is proportional to its base rate times its estimated continuation value, normalized over the C candidates.

Choose one of the candidates at each step

$$\tilde{u}_t(y_c \mid x_t) \;\propto\; u_t(y_c \mid x_t)\,\hat{h}_t(y_c), \qquad c = 1, \ldots, C$$

The selection probability of candidate c is proportional to its base rate times its estimated continuation value, normalized over the C candidates.

At each step we draw C candidate edits from the base rates, score them by rollout, and sample one.

Choose one of the candidates at each step

$$\boxed{\tilde{u}_t(y_c \mid x_t) \;\propto\; u_t(y_c \mid x_t)\,\hat{h}_t(y_c), \qquad c = 1, \ldots, C}$$

The selection probability of candidate c is proportional to its base rate times its estimated continuation value, normalized over the C candidates.

At each step we draw C candidate edits from the base rates, score them by rollout, and sample one.

This is a selection rule over sampled candidates, because a variable-length state space has too many destinations for a full rate.

pCoMole: two feasible molecule edits

Toy edits P and Q have weighted objective gaps (0.4, 0.1) and (0.2, 0.2). Both are feasible and have equal base rates. Take the sum coefficient as 0.1 and reward strength as 10, with deterministic rollouts.

pCoMole: two feasible molecule edits

Toy edits P and Q have weighted objective gaps (0.4, 0.1) and (0.2, 0.2). Both are feasible and have equal base rates. Take the sum coefficient as 0.1 and reward strength as 10, with deterministic rollouts.

$$U(P) = 0.1 + 0.1(0.4 + 0.1) = 0.15, \qquad U(Q) = 0.2 + 0.1(0.2 + 0.2) = 0.24$$

pCoMole: two feasible molecule edits

Toy edits P and Q have weighted objective gaps (0.4, 0.1) and (0.2, 0.2). Both are feasible and have equal base rates. Take the sum coefficient as 0.1 and reward strength as 10, with deterministic rollouts.

$$U(P) = 0.1 + 0.1(0.4 + 0.1) = 0.15, \qquad U(Q) = 0.2 + 0.1(0.2 + 0.2) = 0.24$$

Q earns the larger utility because its weakest weighted objective is stronger.

pCoMole: two feasible molecule edits

Toy edits P and Q have weighted objective gaps (0.4, 0.1) and (0.2, 0.2). Both are feasible and have equal base rates. Take the sum coefficient as 0.1 and reward strength as 10, with deterministic rollouts.

$$U(P) = 0.1 + 0.1(0.4 + 0.1) = 0.15, \qquad U(Q) = 0.2 + 0.1(0.2 + 0.2) = 0.24$$

Q earns the larger utility because its weakest weighted objective is stronger.

$$\Pr(Q \mid P, Q) = \frac{e^{10(0.24)}}{e^{10(0.15)} + e^{10(0.24)}} \approx 0.711$$

pCoMole: two feasible molecule edits

Toy edits P and Q have weighted objective gaps (0.4, 0.1) and (0.2, 0.2). Both are feasible and have equal base rates. Take the sum coefficient as 0.1 and reward strength as 10, with deterministic rollouts.

$$U(P) = 0.1 + 0.1(0.4 + 0.1) = 0.15, \qquad U(Q) = 0.2 + 0.1(0.2 + 0.2) = 0.24$$

Q earns the larger utility because its weakest weighted objective is stronger.

$$\Pr(Q \mid P, Q) = \frac{e^{10(0.24)}}{e^{10(0.15)} + e^{10(0.24)}} \approx 0.711$$

The candidate rule selects Q about 71.1% of the time. Here each rollout value equals its terminal reward.

pCoMole: two feasible molecule edits

Toy edits P and Q have weighted objective gaps (0.4, 0.1) and (0.2, 0.2). Both are feasible and have equal base rates. Take the sum coefficient as 0.1 and reward strength as 10, with deterministic rollouts.

$$U(P) = 0.1 + 0.1(0.4 + 0.1) = 0.15, \qquad U(Q) = 0.2 + 0.1(0.2 + 0.2) = 0.24$$

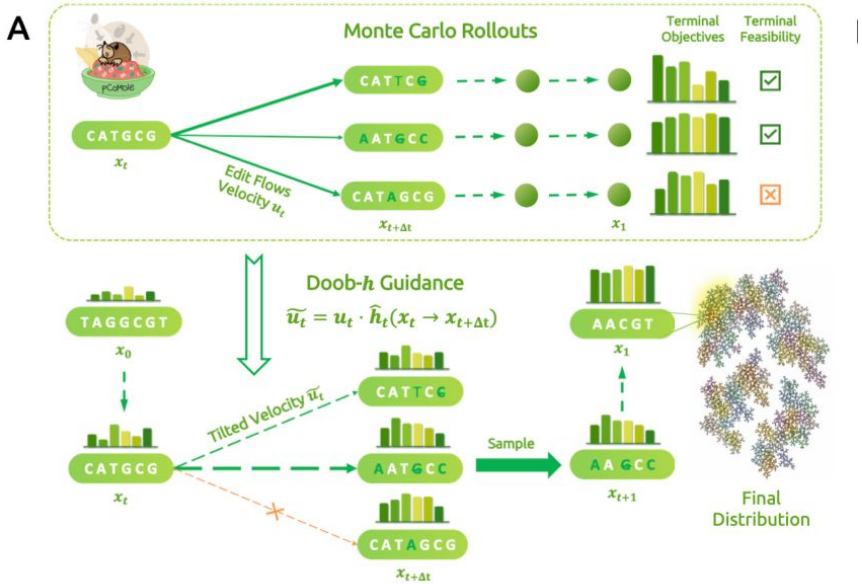
Q earns the larger utility because its weakest weighted objective is stronger.

$$\Pr(Q \mid P, Q) = \frac{e^{10(0.24)}}{e^{10(0.15)} + e^{10(0.24)}} \approx 0.711$$

The candidate rule selects Q about 71.1% of the time. Here each rollout value equals its terminal reward.

We calculate utility from the worst objective, then exponentiate it to obtain the sampling weight.

In pCoMole, we score candidate edits by short rollouts and tilt toward the best feasible ending



Candidate edits from the base velocity, rollouts for terminal utility, then a tilted velocity.

Chen,..,Chatterjee. *NeurIPS* (2026)

Shrinking a fluorescent protein to a fixed length

Method	$\lvert\lambda_{\text{exc}} - 488\rvert$ (↓)	Brightness
RayGun	22.3594	9.4207
SCISOR	20.1279	19.0333
pCoMole (UniRef$_S$ Edit Flow)	**5.8213**	43.5116
pCoMole (GFP Edit Flow)	8.4372	**49.0121**

Chen,..,Chatterjee. *NeurIPS* (2026)

The sequence-model baseline misses the target excitation wavelength by 22.4 nanometres

Method	$\lvert\lambda_{\text{exc}} - 488\rvert$ (↓)	Brightness
RayGun	22.3594	9.4207
SCISOR	20.1279	19.0333
pCoMole (UniRef$_S$ Edit Flow)	**5.8213**	43.5116
pCoMole (GFP Edit Flow)	8.4372	**49.0121**

Chen,..,Chatterjee. *NeurIPS* (2026)

The second baseline misses by 20.1 nanometres and has brightness 19.0

Method	$\lvert\lambda_{\text{exc}} - 488\rvert$ (↓)	Brightness
RayGun	22.3594	9.4207
SCISOR	20.1279	19.0333
pCoMole (UniRef$_S$ Edit Flow)	**5.8213**	43.5116
pCoMole (GFP Edit Flow)	8.4372	**49.0121**

Chen,..,Chatterjee. *NeurIPS* (2026)

With pCoMole, the wavelength error is 5.8 nanometres and brightness is 43.5 at the same length of 213 residues

Method	$\lvert\lambda_{\text{exc}} - 488\rvert$ (↓)	Brightness
RayGun	22.3594	9.4207
SCISOR	20.1279	19.0333
pCoMole (UniRef$_S$ Edit Flow)	**5.8213**	43.5116
pCoMole (GFP Edit Flow)	8.4372	**49.0121**

Chen,..,Chatterjee. *NeurIPS* (2026)

Trained on fluorescent proteins instead, it reaches brightness 49.0 at 8.4 nanometres

Method	$\lvert\lambda_{\text{exc}} - 488\rvert$ (↓)	Brightness
RayGun	22.3594	9.4207
SCISOR	20.1279	19.0333
pCoMole (UniRef$_S$ Edit Flow)	**5.8213**	43.5116
pCoMole (GFP Edit Flow)	8.4372	**49.0121**

Chen,..,Chatterjee. *NeurIPS* (2026)

Let’s add guidance to our sampler

```python
import torch
import torch.nn.functional as F

def guided_step(model, prop, xt, t, h, gamma=1.0):
    oh = F.one_hot(xt, V).float().requires_grad_()
    logp = prop(oh, t).sum()                    # property on
noisy x
    g = torch.autograd.grad(logp, oh)[0]        # one backward
pass
    adv = g - (oh * g).sum(-1, keepdim=True)    # Taylor ratio
    with torch.no_grad():
        p1 = model(xt, t).softmax(-1)           # denoiser
        rate = kd(t) / (1 - k(t)).clamp(min=1e-4)
        u = rate * p1 * torch.exp(gamma * adv)  # every rate
tilted
        u.scatter_(-1, xt[..., None], 0.0)      # no
self-transition
        jump = torch.rand(xt.shape) < h * u.sum(-1)
        y = torch.distributions.Categorical(probs=u).sample()
    return torch.where(jump, y, xt)
```

We pass the one-hot noisy sequence to the property model

- We differentiate with respect to the one-hot encoding to obtain a gradient component for every letter.

```
import torch
import torch.nn.functional as F

def guided_step(model, prop, xt, t, h, gamma=1.0):
    oh = F.one_hot(xt, V).float().requires_grad_()
    logp = prop(oh, t).sum()                        # property on
noisy x
    g = torch.autograd.grad(logp, oh)[0]            # one backward
pass
    adv = g - (oh * g).sum(-1, keepdim=True)        # Taylor ratio
    with torch.no_grad():
        p1 = model(xt, t).softmax(-1)               # denoiser
        rate = kd(t) / (1 - k(t)).clamp(min=1e-4)
        u = rate * p1 * torch.exp(gamma * adv)      # every rate
tilted
        u.scatter_(-1, xt[..., None], 0.0)          # no
self-transition
        jump = torch.rand(xt.shape) < h * u.sum(-1)
        y = torch.distributions.Categorical(probs=u).sample()
    return torch.where(jump, y, xt)
```

One gradient replaces a score for every destination

- We differentiate with respect to the one-hot encoding to obtain a gradient component for every letter.
- Subtract the gradient at the letter currently in place, which is the Taylor ratio.

```
import torch
import torch.nn.functional as F

def guided_step(model, prop, xt, t, h, gamma=1.0):
    oh = F.one_hot(xt, V).float().requires_grad_()
    logp = prop(oh, t).sum()                        # property on
noisy x
    g = torch.autograd.grad(logp, oh)[0]            # one backward
pass
    adv = g - (oh * g).sum(-1, keepdim=True)        # Taylor ratio
    with torch.no_grad():
        p1 = model(xt, t).softmax(-1)               # denoiser
        rate = kd(t) / (1 - k(t)).clamp(min=1e-4)
        u = rate * p1 * torch.exp(gamma * adv)      # every rate
tilted
        u.scatter_(-1, xt[..., None], 0.0)          # no
self-transition
        jump = torch.rand(xt.shape) < h * u.sum(-1)
        y = torch.distributions.Categorical(probs=u).sample()
    return torch.where(jump, y, xt)
```

The unguided rates come from the denoiser, as before

- Nothing about the trained model changes.

```
import torch
import torch.nn.functional as F

def guided_step(model, prop, xt, t, h, gamma=1.0):
    oh = F.one_hot(xt, V).float().requires_grad_()
    logp = prop(oh, t).sum()                    # property on
noisy x
    g = torch.autograd.grad(logp, oh)[0]        # one backward
pass
    adv = g - (oh * g).sum(-1, keepdim=True)    # Taylor ratio
    with torch.no_grad():
        p1 = model(xt, t).softmax(-1)           # denoiser
        rate = kd(t) / (1 - k(t)).clamp(min=1e-4)
        u = rate * p1 * torch.exp(gamma * adv)  # every rate
tilted
        u.scatter_(-1, xt[..., None], 0.0)      # no
self-transition
        jump = torch.rand(xt.shape) < h * u.sum(-1)
        y = torch.distributions.Categorical(probs=u).sample()
    return torch.where(jump, y, xt)
```

We multiply each rate by its guidance factor

- Nothing about the trained model changes.
- At γ = 1, we recover the exact conditional process.

```python
import torch
import torch.nn.functional as F

def guided_step(model, prop, xt, t, h, gamma=1.0):
    oh = F.one_hot(xt, V).float().requires_grad_()
    logp = prop(oh, t).sum()                    # property on
noisy x
    g = torch.autograd.grad(logp, oh)[0]        # one backward
pass
    adv = g - (oh * g).sum(-1, keepdim=True)    # Taylor ratio
    with torch.no_grad():
        p1 = model(xt, t).softmax(-1)           # denoiser
        rate = kd(t) / (1 - k(t)).clamp(min=1e-4)
        u = rate * p1 * torch.exp(gamma * adv)  # every rate
tilted
        u.scatter_(-1, xt[..., None], 0.0)      # no
self-transition
        jump = torch.rand(xt.shape) < h * u.sum(-1)
        y = torch.distributions.Categorical(probs=u).sample()
    return torch.where(jump, y, xt)
```

We multiply each rate by its guidance factor

- Nothing about the trained model changes.
- At γ = 1, we recover the exact conditional process. Larger γ increases the preference for the desired property.

```python
import torch
import torch.nn.functional as F

def guided_step(model, prop, xt, t, h, gamma=1.0):
    oh = F.one_hot(xt, V).float().requires_grad_()
    logp = prop(oh, t).sum()                    # property on
noisy x
    g = torch.autograd.grad(logp, oh)[0]        # one backward
pass
    adv = g - (oh * g).sum(-1, keepdim=True)    # Taylor ratio
    with torch.no_grad():
        p1 = model(xt, t).softmax(-1)           # denoiser
        rate = kd(t) / (1 - k(t)).clamp(min=1e-4)
        u = rate * p1 * torch.exp(gamma * adv)  # every rate
tilted
        u.scatter_(-1, xt[..., None], 0.0)      # no
self-transition
        jump = torch.rand(xt.shape) < h * u.sum(-1)
        y = torch.distributions.Categorical(probs=u).sample()
    return torch.where(jump, y, xt)
```

The rest of the step is unchanged

- Nothing about the trained model changes.
- At γ = 1, we recover the exact conditional process. Larger γ increases the preference for the desired property.

```python
import torch
import torch.nn.functional as F

def guided_step(model, prop, xt, t, h, gamma=1.0):
    oh = F.one_hot(xt, V).float().requires_grad_()
    logp = prop(oh, t).sum()                    # property on
noisy x
    g = torch.autograd.grad(logp, oh)[0]        # one backward
pass
    adv = g - (oh * g).sum(-1, keepdim=True)    # Taylor ratio
    with torch.no_grad():
        p1 = model(xt, t).softmax(-1)           # denoiser
        rate = kd(t) / (1 - k(t)).clamp(min=1e-4)
        u = rate * p1 * torch.exp(gamma * adv)  # every rate
tilted
        u.scatter_(-1, xt[..., None], 0.0)      # no
self-transition
        jump = torch.rand(xt.shape) < h * u.sum(-1)
        y = torch.distributions.Categorical(probs=u).sample()
    return torch.where(jump, y, xt)
```

Summary: several ways to steer one model

Summary: several ways to steer one model

Guidance is a Doob h-transform:

$$u_t^h(y, x) = \frac{h_t(y)}{h_t(x)}\, u_t(y, x), \qquad h_t(x) = \mathbb{E}\big[G(X_1) \mid X_t = x\big]$$

Summary: several ways to steer one model

Guidance is a Doob h-transform:

$$u_t^h(y, x) = \frac{h_t(y)}{h_t(x)}\, u_t(y, x), \qquad h_t(x) = \mathbb{E}\big[G(X_1) \mid X_t = x\big]$$

The guided rate multiplies the base rate by the ratio of the expected terminal reward at the destination to the one at the current state.

Summary: several ways to steer one model

Guidance is a Doob h-transform:

$$u_t^h(y, x) = \frac{h_t(y)}{h_t(x)}\, u_t(y, x), \qquad h_t(x) = \mathbb{E}\big[G(X_1) \mid X_t = x\big]$$

For Discrete Guidance, G is a property likelihood. For pCoMole, G combines exponentiated utility with a feasibility indicator.

Summary: several ways to steer one model

Guidance is a Doob h-transform:

$$u_t^h(y, x) = \frac{h_t(y)}{h_t(x)}\, u_t(y, x), \qquad h_t(x) = \mathbb{E}\big[G(X_1) \mid X_t = x\big]$$

For Discrete Guidance, G is a property likelihood. For pCoMole, G combines exponentiated utility with a feasibility indicator.

In MOG-DFM, we use rank and direction scores to reduce computation, at the cost of exact conditioning.

Summary: several ways to steer one model

Guidance is a Doob h-transform:

$$u_t^h(y, x) = \frac{h_t(y)}{h_t(x)}\, u_t(y, x), \qquad h_t(x) = \mathbb{E}\big[G(X_1) \mid X_t = x\big]$$

For Discrete Guidance, G is a property likelihood. For pCoMole, G combines exponentiated utility with a feasibility indicator.

In MOG-DFM, we use rank and direction scores to reduce computation, at the cost of exact conditioning.

We increase guidance strength with γ. Only γ = 1 samples the conditional distribution exactly.

Every sampler so far takes hundreds of steps

Every sampler so far takes hundreds of steps

In Discrete Flow Matching we took 1024 steps for text, in metric paths 100 for images, and in Edit Flows up to 10,000 for code.

Every sampler so far takes hundreds of steps

In Discrete Flow Matching we took 1024 steps for text, in metric paths 100 for images, and in Edit Flows up to 10,000 for code.

An autoregressive model of the same size takes one pass per token, so the speedup has to come from the step count.

Every sampler so far takes hundreds of steps

In Discrete Flow Matching we took 1024 steps for text, in metric paths 100 for images, and in Edit Flows up to 10,000 for code.

An autoregressive model of the same size takes one pass per token, so the speedup has to come from the step count.

What prevents one-step discrete generation? How can we change the model to address that limitation?

Could a better pairing make the trajectories straighter and reduce the number of sampling steps?